Active Fault-Tolerant Control of a Quadrotor with a Total Rotor Failure: Re-implementation on X500 Parameters

1 Introduction

This report summarises a Python re-implementation of the paper *"Active fault-tolerant control for quadrotors subjected to a complete rotor failure"* by Lu and van Kampen (IROS 2015). The paper builds a complete system for the case where one rotor stops working completely. It has two parts:

- **FDI** (fault detection and isolation): it finds out that a rotor has failed and which one.
- **FTC** (fault-tolerant control): it keeps the quadrotor flying using only the three remaining rotors.

We reproduce this with the parameters of the PX4 Gazebo X500 drone, but in a pure Python simulation (no Gazebo yet).

2 How the Method Works

2.1 Detecting the failed rotor

Each rotor i has a loss-of-effectiveness (LOE) factor l_i: $l_i = 0$ means healthy and $l_i = 1$ means completely dead. The total thrust T and the three torques τ_x, τ_y, τ_z are estimated from the measured motion (gyro and vertical acceleration). They are then compared with what the motor commands should have produced:

$$\hat{\mathbf{l}} = \mathbf{1} - \mathbf{A}^{-1} \begin{bmatrix} \hat{T} & \hat{\tau}_x & \hat{\tau}_y & \hat{\tau}_z \end{bmatrix}^T . \tag{1}$$

If any $\hat{l}_i$ stays above a threshold l_T, that rotor is declared failed.

2.2 Flying with three rotors

Three rotors cannot control roll, pitch, yaw and thrust together, so the paper gives up yaw control. The quadrotor spins about its vertical axis and only the direction of the thrust axis is controlled:

- An outer loop turns the desired position into a desired thrust direction.
- An inner loop (INDI, incremental nonlinear dynamic inversion) tracks the angular rates p, q using measured angular acceleration.
- A reduced control allocation sends thrust and the two torques to the three working rotors.

3 Implementation Setup

All dynamics follow the paper's model (Section II). The parameters come from the X500 SDF:

Quantity	Paper	This work (X500)
Mass	not stated	2.0 kg
Inertia $I_x = I_y$	6.2×10^{-3} kg m^2	0.02167 kg m^2
Inertia I_z	1.2×10^{-2} kg m^2	0.04 kg m^2
Arm length	0.232 m (plus layout)	0.174 m per axis (X layout)
Thrust coefficient	3.13×10^{-5}	8.55×10^{-6}
Torque / thrust ratio	$d/b \approx 0.024$	0.016
Motor time constant	0.02 s	0.0125 s up, 0.025 s down
Fault	rotor 2 at $t = 30$ s	rotor 0 at $t = 10$ s

Table 1: Main parameters.

The simulation uses RK4 integration at 2 ms. Gyro noise is taken from the SDF IMU noise (8.7×10^{-4} rad/s), and vertical velocity has 0.01 m/s noise. The reference is a smooth 3D path. The fault is injected by forcing one rotor to zero speed.

4 Comparison with the Paper

Item	Paper	This implementation
Platform	Numerical simulation, no hardware	Python simulation, same style
Frame layout	Plus configuration	X configuration, new allocation matrix
Thrust model	Blade-element coefficients with factor η_i	Plain $k_f\omega^2$, so $\eta_i = 1$
Hub forces	Included in the plant	Left out
Fault detection	Estimated $\hat{l}_i$ against threshold	Same, plus 10 consecutive samples above threshold
Delay matching	Not discussed	Motor-speed model passed through the same filter as the gyro derivative
Thrust estimate	Needs $\hat{\phi}, \hat{\theta}, \dot{w}$ from an optic track system	Attitude and noisy vertical velocity used directly
Inner loop	$\tau_c = I(v - \dot{\omega}_0)$	$\tau_c = \tau_0 + I(v - \dot{\omega}_0)$ (full INDI form)
Nominal controller	Omitted from the paper	Own simple cascaded INDI controller
Yaw drag k_r	6×10^{-3}	10^{-2} (placeholder, not identified)

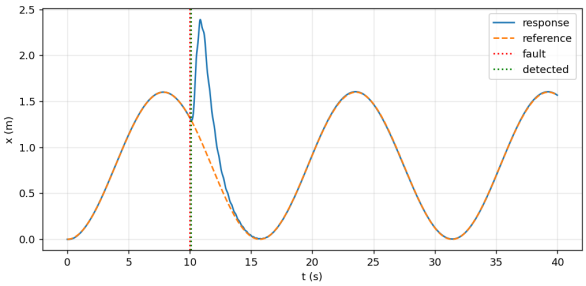
Table 2: Main differences.

5 Results

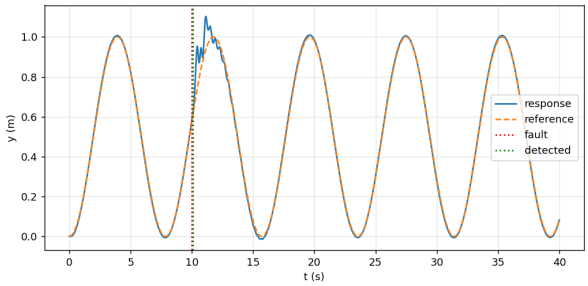
Figures 1–2 show one run with rotor 0 failing at $t = 10$ s and the motor limit set to 1500 rad/s.

- The failed rotor is detected correctly within about 74 ms of the fault (the paper reports about 30 ms).

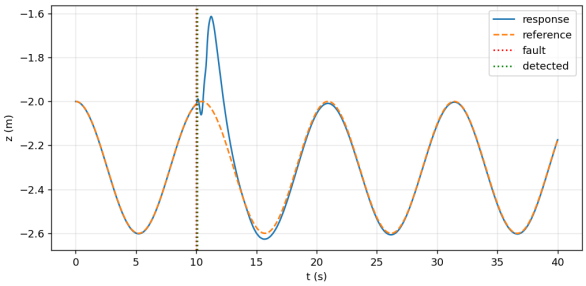
- After switching to the fault-tolerant controller, the drone tracks the reference with position errors of a few millimetres.
- The yaw rate settles near -30 rad/s. The vehicle keeps spinning, as expected.



(a) x position



(b) y position



(c) z position

Figure 1: Position response against the reference. The fault and detection times are marked.

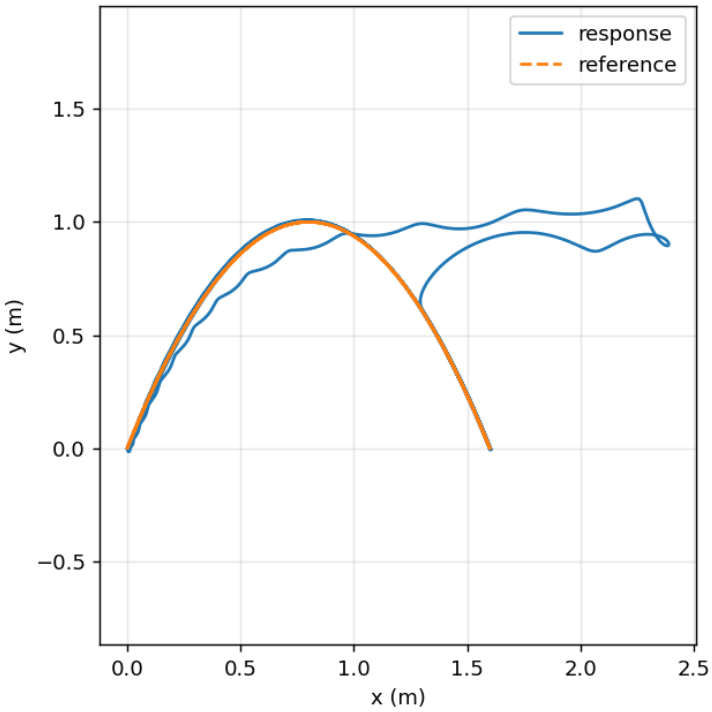


Figure 2: Horizontal path against the reference path.

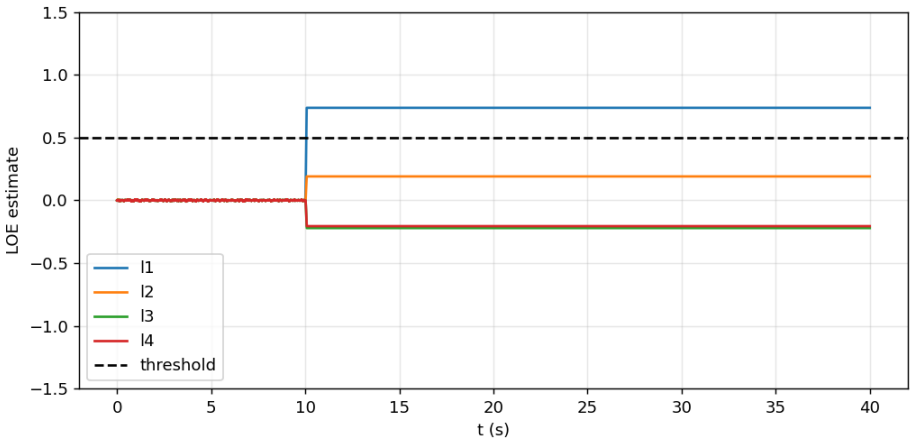


Figure 3: Estimated LOE factors. Only the failed rotor rises above the threshold.

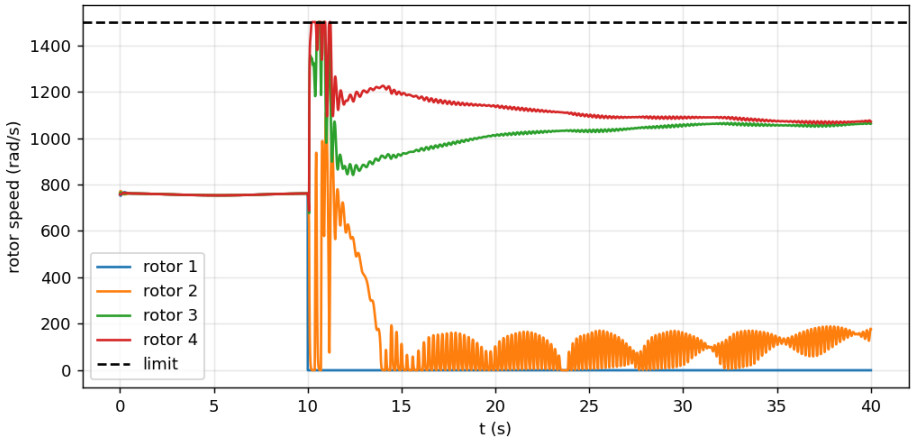


Figure 4: Rotor speeds. The failed rotor drops to zero and the others increase.

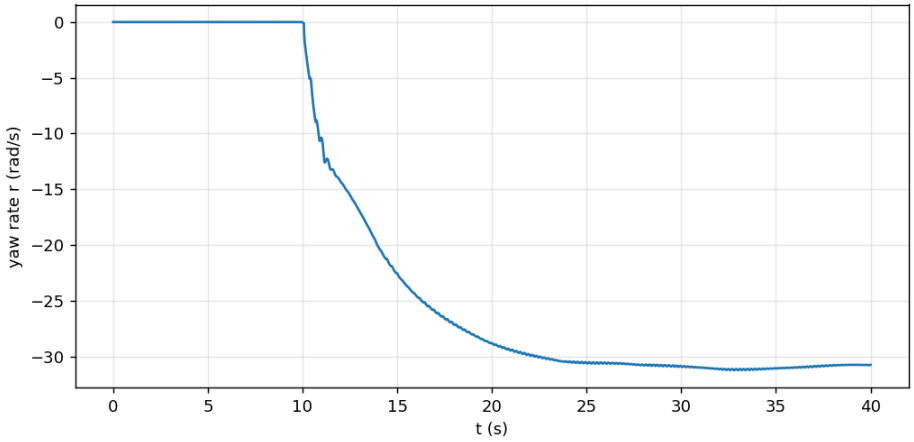


Figure 5: Yaw rate after the failure. Yaw is not controlled, so the vehicle spins.

6 Thrust Limit Problem

The paper's equilibrium keeps the roll and pitch torques at zero. With rotor 0 dead this gives $T_1 = 0$ and $T_2 = T_3 = mg/2 = 9.81$ N, so the two diagonal rotors carry the whole weight. The stock X500 limit of 1000 rad/s gives only $k_f \omega_{max}^2 = 8.55$ N per rotor, so the motors saturate and the drone loses altitude.

Max rotor speed (rad/s)	Result
1000 (stock)	Diverges, motors saturate
1200	Recovers, errors of a few cm
1500	Recovers, errors of a few mm

Table 3: Effect of the motor speed limit.

The minimum speed for static hover is $\omega \geq \sqrt{9.81/8.54858 \times 10^{-6}} \approx 1071$ rad/s. The paper's

quadrotor had plenty of thrust margin, so it never mentions this limit.

7 Limitations and Next Steps

- The yaw drag k_r is a guess and sets the spin rate. It should be identified from the simulator.
- Hub forces and the blade-element thrust model are not included.
- The threshold l_T was fixed at 0.5 and not tuned on fault-free runs as the paper suggests.
- The next step is to port the controller to Gazebo with the X500 model and raise the motor limit to at least 1200 rad/s.

8 Conclusion

The main idea of the paper works on X500 parameters: the failed rotor is detected quickly and the quadrotor keeps tracking its reference using three rotors. The main extra finding is that the X500's stock motor speed limit is too low for this scheme, so it must be raised or the vehicle made lighter.